

Plane strain analysis

Luciano Jacinto

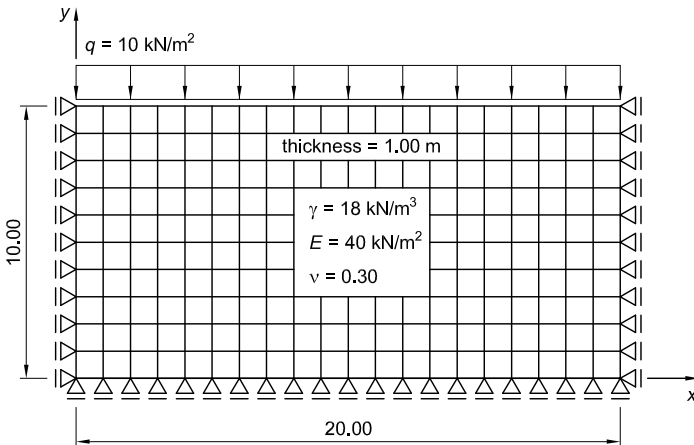
April 18, 2026

1 Purpose

- This sample illustrates how to build and solve a plane strain problem using NextFEMpy module. It is very common to use the theory of elasticity with elements in a plane strain state in geotechnical problems.
- The model must be defined in the XY plane (according to NextFEM rules.) The key methods to be used are:
 - `setPlaneStrainElement(ele, isPlaneStrain=True)`, which sets a regular quad element into a plane strain element.
 - `setSelfWeightDirection(direction=-2)`, which sets the gravity direction ($-Z$, by default) to the $-Y$ direction.

2 Model description

- The model consists of a soil mass subject to a plane strain state:



The soil mass is subject to two loads: (1) weight of the soil, g , and (2) a uniform load, q , on its top.

- According to the theory of elasticity, for a plane strain state, the earth pressure coefficient¹ relates to Poisson ratio by the formula:

$$k = \frac{\nu}{1 - \nu}$$

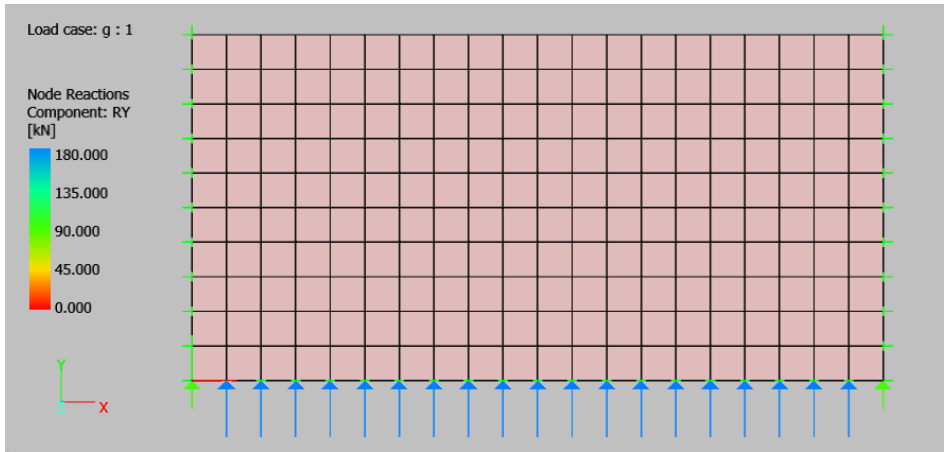
$$= \frac{0.30}{1 - 0.30} = 0.4286 \approx 0.43.$$

- Since in this example it is easy to calculate vertical pressures and the soil elements suffers no distortion, it will be possible to calculate horizontal pressures through the earth pressure coefficient k .

3 Results

3.1 Reactions

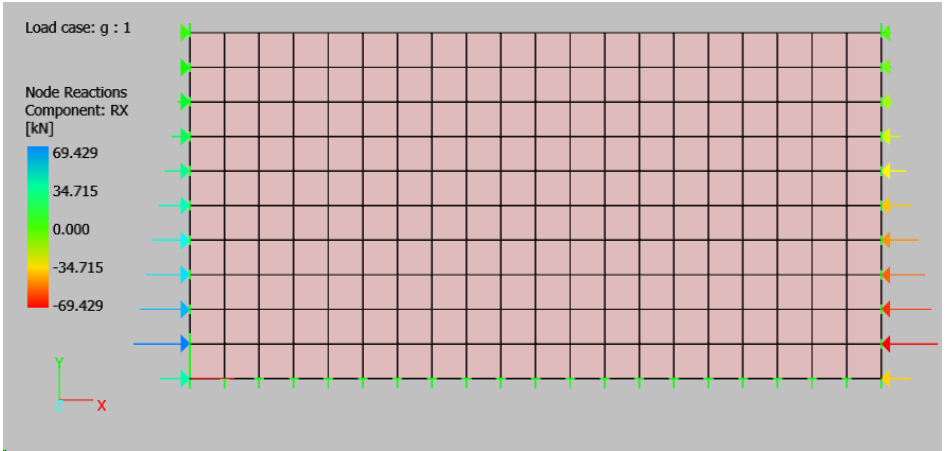
- Vertical reactions due to self-weight:



Hand calculation: $R = (18)(10.0)(1.00)(1.00) = 180.00 \text{ kN}$.

- Horizontal reactions due to self-weight:

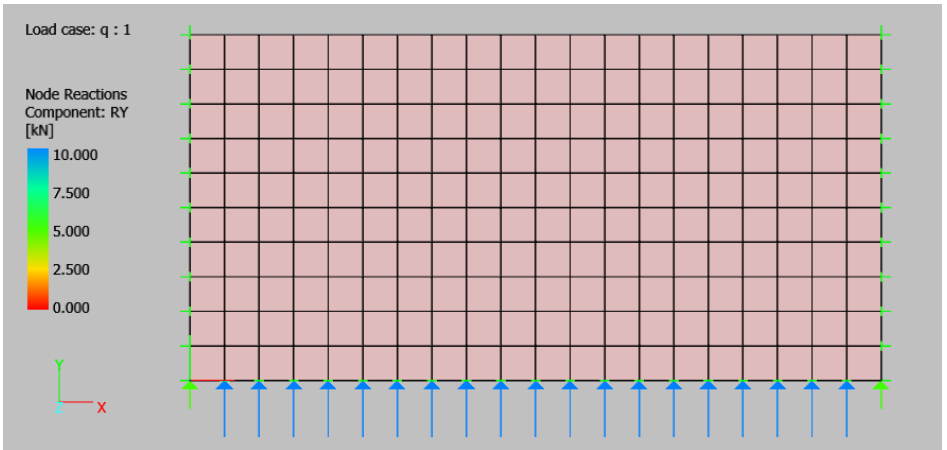
¹Ratio between horizontal and vertical pressures ($k = p_h/p_v$).



The maximum reaction occur at the first level of horizontal supports. NextFEM gives $R = 69.429$ kN.

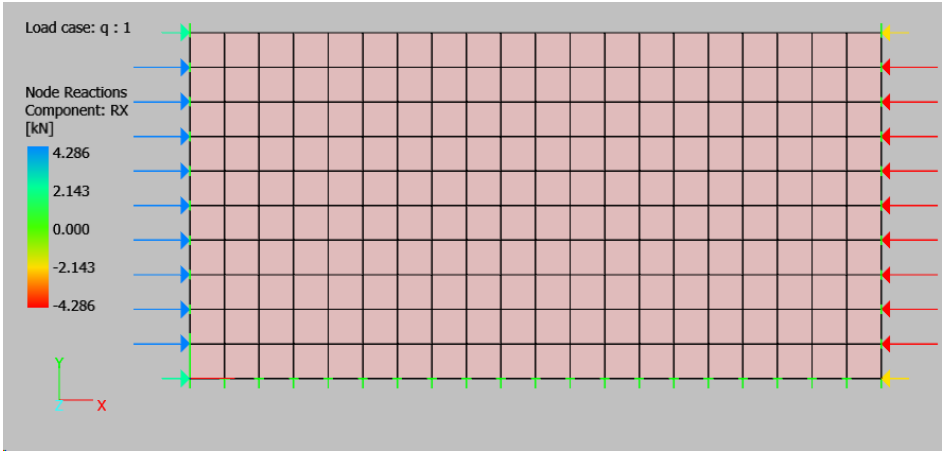
Hand calculation: $R = (0.4286)(18)(9.00)(1.00)(1.00) = 69.43$ kN.

- Vertical reactions due to q load:



Hand calculation: $R = (10.0)(1.00)(1.00) = 10.0$ kN.

- Horizontal reactions due to q load:

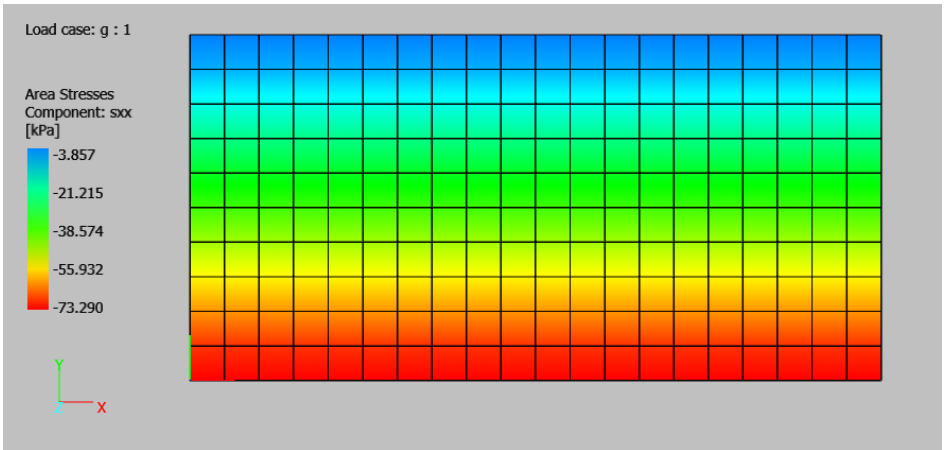


The horizontal reactions due to q load are constant. NextFEM gives $R = 4.286$ kN.

Hand calculation: $R = (0.4286)(10.00)(1.00)(1.00) = 4.29$ kN.

3.2 Horizontal pressures

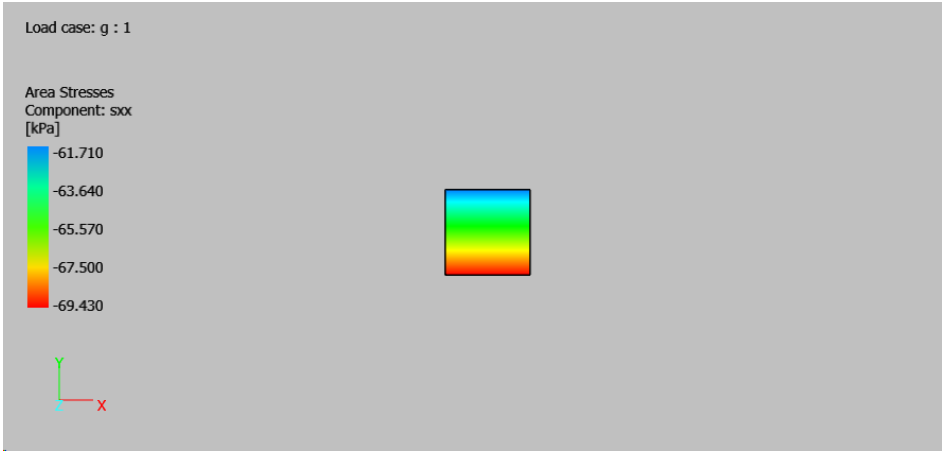
- Horizontal pressure due to self-weight:



Hand calculation: $p_h = (0.4286)(18)(10.0) = 77.15$ kN/m².

NextFEM gives 73.29 kN/m², which represents an error of -5.0 %.

This result refers to a boundary element (at the base). Let's verify for an element immediately above of that:



According to this result, NextFEM gives:

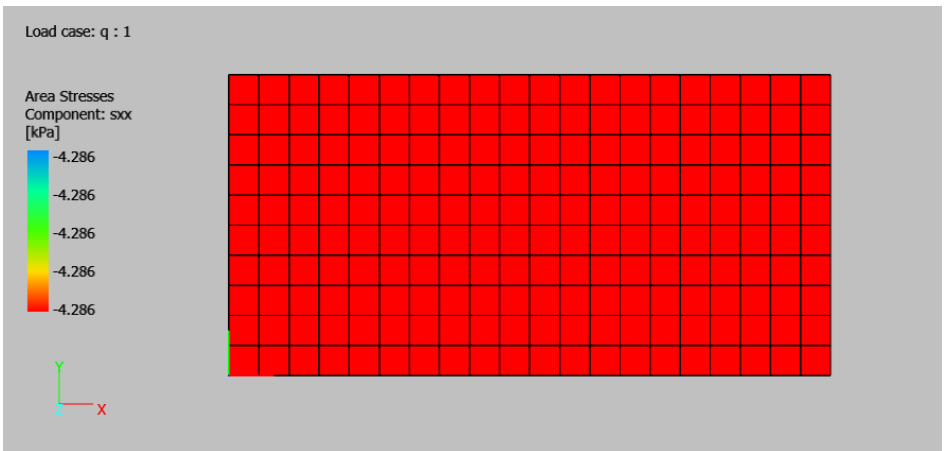
- Top of the element: $p_h = 61.71 \text{ kN/m}^2$
- Bottom of the element: $p_h = 69.43 \text{ kN/m}^2$.

Hand calculations:

- Top of the element: $p_h = (0.4286)(18)(8.0) = 61.72 \text{ kN/m}^2$
- Bottom of the element: $p_h = (0.4286)(18)(9.0) = 69.43 \text{ kN/m}^2$.

So, for an element that is not a boundary element, we get the exact values.

- Horizontal pressure due to q load:



Hand calculation: $p_h = (0.4286)(10.0) = 4.29 \text{ kN/m}^2$.

4 Python script

- File name: plane_strain_analysis.py
- Versions of the program used when this script was executed:
 - Main program: 2.7.3.2
 - NextFEMpy module: 0.4.3

```
'''
This sample illustrates how to build and solve a plane strain problem.
It is very common to use the theory of elasticity with elements in a
plane strain state in geotechnical problems.

The model consists of a soil mass with a length of 20 m and a height of 10 m.
It is subject to its self-weight and a load of 10 kN/m2 on its top.

The problem is defined in the XY plane, which is mandatory in NextFEM
for this kind of problem.
'''

from nextfempy import NextFEMrest

# Connect to the open instance of NextFEM Designer and create the nf object
nf = NextFEMrest()

# Clear the model
nf.newModel()

# Set units
nf.setUnits('m', 'kN')

# Materials
solo = nf.addIsoMaterial(name='solo', E=40e3, ni=0.30, Wden=18)

# Thickness of the quad elements
sec = nf.addPlanarSection(t=1.00)

# Nodes
n1 = nf.addNode( 0.0,  0.0, 0.0)
n2 = nf.addNode(20.0,  0.0, 0.0)
n3 = nf.addNode( 0.0, 10.0, 0.0)
n4 = nf.addNode(20.0, 10.0, 0.0)

# Macro quad element
A = nf.addQuad(n1, n2, n4, n3, sect=sec, mat=solo)

# Generate the mesh
quads = nf.divideQuad(quadID=A, divX=10, divY=20)

# Set elements as plane strain elements
for ele in quads:
```

```

    nf.setPlaneStrainElement(ele, isPlaneStrain=True)

# Boundary conditions
    blkd = True; free = False

    nf.setBC(n1, blkd, blkd, blkd, free, free, free)
    nf.setBC(n2, blkd, blkd, blkd, free, free, free)

    nf.setBC(n3, blkd, free, blkd, free, free, free)
    nf.setBC(n4, blkd, free, blkd, free, free, free)

    for n in range(5, 23+1, 1):
        nf.setBC(str(n), free, blkd, blkd, free, free, free)

    for n in range(24, 192+1, 21):
        nf.setBC(str(n), blkd, free, blkd, free, free, free)

    for n in range(44, 212+1, 21):
        nf.setBC(str(n), blkd, free, blkd, free, free, free)

# g load
    nf.addLoadCase('g')
    nf.setSelfWeightDirection(direction=-2)
    nf.setSelfWeight('g')

# q load
    nf.addLoadCase('q')
    for ele in range(182, 201+1, 1):
        nf.addEdgeLoad(
            elem = str(ele),
            values = [-10.0],
            edge = 3,
            direction = 2,
            loadcase = 'q',
            local = False
        )

# Analyse the model
    nf.RunModel(outOfProc=True)

# Refresh NextFEM GUI
    nf.refreshDesignerView(0, resize=True)

```